



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

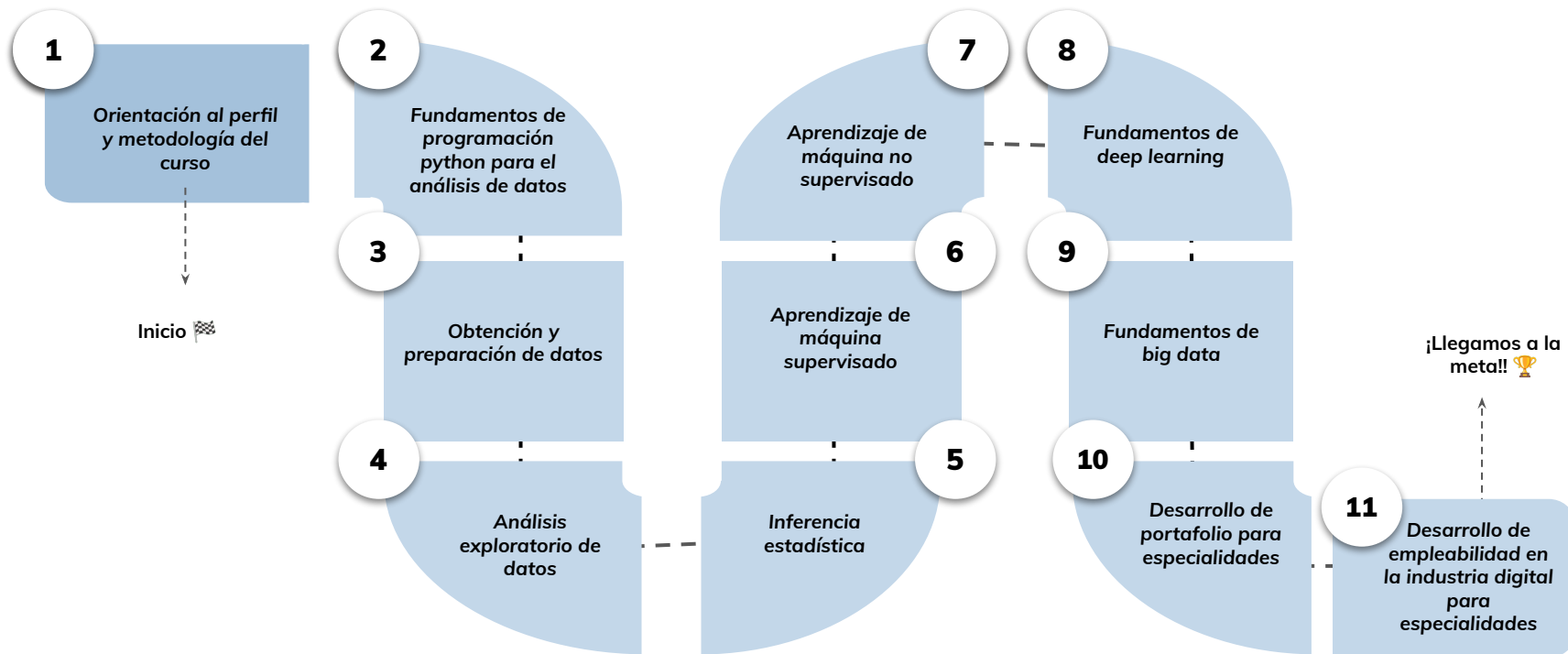


Redes neuronales convolutivas- › Parte I

Aprendizaje Esperado 4: Implementar un modelo predictivo utilizando redes neuronales convolutivas para el reconocimiento de imágenes en python.

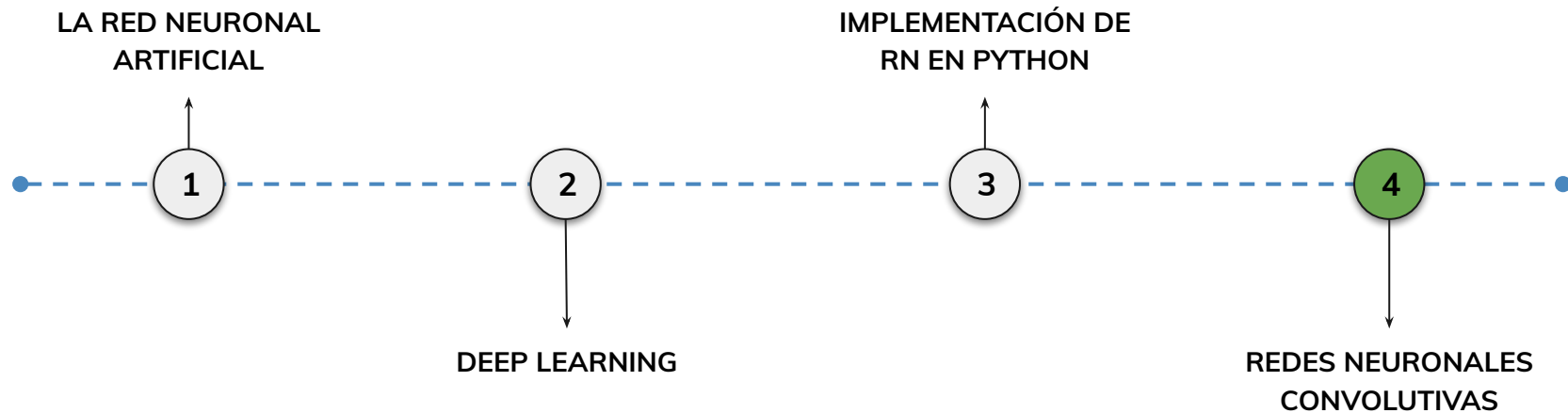
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

1.

Fundamentos de redes neuronales convolutivas (CNN)

Introduciremos los conceptos clave de las redes neuronales convolutivas, sus componentes, ventajas y aplicaciones, preparando el terreno para implementarlas en Python.

Fundamentos y arquitectura de las CNN

¿Qué son las CNN y por qué se utilizan en procesamiento de imágenes?

Ventajas de las CNN frente a redes densas.

Aplicaciones y métricas en CNN

Principales casos de uso: clasificación, detección de objetos, segmentación.

Métricas para evaluar CNN en tareas de visión por computadora: accuracy, precision, recall, F1-score.

Objetivos de aprendizaje

¿Qué aprenderás?

- Comprender qué es una red neuronal convolutiva y sus diferencias con redes densas.
- Conocer los componentes clave de una CNN y su función dentro de la arquitectura.
- Identificar las ventajas y limitaciones de las CNN en el reconocimiento de imágenes.
- Reconocer las principales aplicaciones de las CNN en visión por computadora.
- Analizar métricas de evaluación para medir el rendimiento de una CNN.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Entrenamos y evaluamos redes neuronales artificiales para regresión y clasificación en Python.
- Aplicamos técnicas de regularización y validación para mejorar la generalización de los modelos.
- Ajustamos hiperparámetros y analizamos métricas de desempeño.
- Conocimos el Transfer Learning y cómo reutilizar modelos preentrenados.

Redes neuronales convolutivas



Redes Neuronales Convolutivas: Fundamentos e Implementación

Las redes neuronales convolutivas (CNN, por sus siglas en inglés) han transformado el campo del procesamiento de imágenes y visión por computadora. Inspiradas en el funcionamiento del sistema visual de los seres vivos, estas redes están especialmente diseñadas para reconocer patrones espaciales en imágenes mediante una combinación de operaciones matemáticas conocidas como convoluciones, funciones de activación no lineales, y reducción de dimensionalidad.

Objetivo del Manual

Este manual tiene como objetivo brindar una comprensión teórica y práctica de las CNN, desde sus fundamentos hasta su implementación con Python utilizando librerías como Keras. Se abordarán los elementos fundamentales de la arquitectura convolutiva, sus etapas y su aplicabilidad en problemas reales, así como estrategias de evaluación y optimización.



Con estas herramientas, podrás crear modelos capaces de identificar objetos, clasificar imágenes y resolver problemas visuales con alto grado de precisión.

¿Qué es una red neuronal convolutiva?

Una red neuronal convolutiva (CNN) es una arquitectura de red especializada en el procesamiento de datos con una estructura de cuadrícula, como las imágenes. A diferencia de las redes neuronales densas (fully connected), que procesan los datos como vectores planos, las CNN mantienen la estructura espacial de los datos, lo que permite detectar patrones locales como bordes, texturas o formas.

Componentes de una CNN

Capas Convolutivas

Realizan operaciones convolutivas para extraer características espaciales de las imágenes.

Funciones de Activación

Introducen no linealidad al modelo, permitiendo aprender patrones complejos.

Capas de Pooling

Reducen la dimensionalidad manteniendo la información más relevante.

Capas Densas

Realizan la clasificación final basada en las características extraídas.

Ventajas de las CNN



Reducción del número de parámetros

Gracias al uso de filtros compartidos, las CNN requieren menos parámetros que las redes completamente conectadas.



Detección jerárquica de características

Permiten identificar patrones desde bajo nivel (bordes) hasta alto nivel (objetos completos).



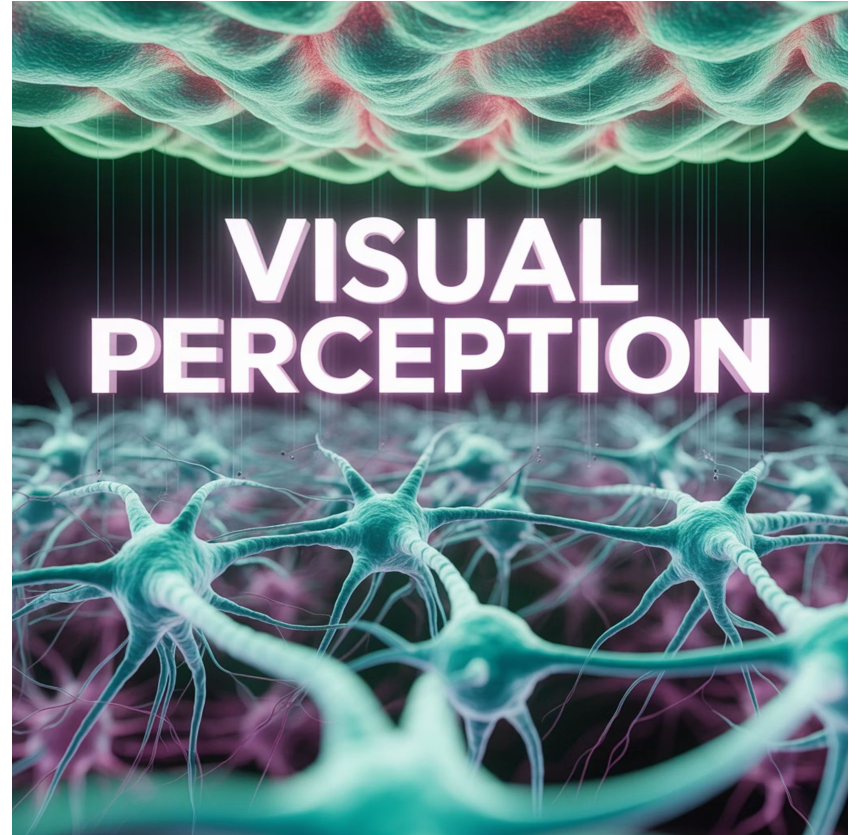
Capacidad de generalización

Funcionan bien con datos visuales complejos y variados, generalizando patrones importantes.

Inspiración Biológica

La arquitectura de las CNN se inspira en el córtex visual humano, donde ciertas neuronas responden a patrones específicos en regiones específicas del campo visual.

Esta similitud con el sistema visual biológico es una de las razones por las que las CNN son tan efectivas en tareas de procesamiento de imágenes.



¿Qué es una convolución?

La **convolución** es una operación matemática que combina dos funciones para producir una tercera. En el contexto de las CNN, se utiliza para aplicar filtros o kernels sobre una imagen, con el fin de extraer características específicas como bordes, esquinas o texturas.

Proceso de Convolución

Por ejemplo, al aplicar un filtro de 3x3 sobre una imagen, se multiplica cada valor del filtro con los valores correspondientes de un fragmento de la imagen, y luego se suman los productos. Este proceso se repite desplazando el filtro por toda la imagen (stride), generando un nuevo mapa de características.

Elementos de la Convolución

Concepto	Descripción
Imagen de entrada	Matriz de pixeles
Filtro (kernel)	Matriz pequeña (ej. 3x3 o 5x5)
Resultado	Mapa de características (feature map)

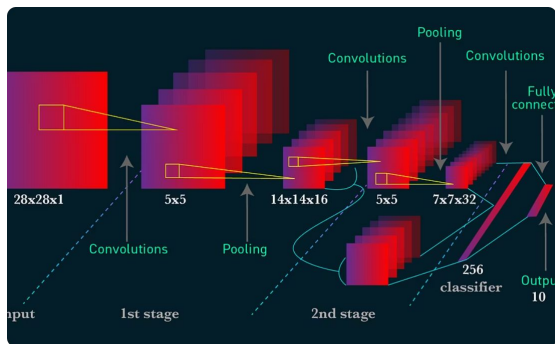
Ventajas de la Convolución

La convolución permite extraer características locales sin necesidad de aplanar la imagen completa, lo cual conserva información espacial importante. A través de múltiples capas convolutivas, se pueden detectar patrones de bajo nivel (bordes) y patrones de alto nivel (rostros, objetos).

Campo de utilización de las redes neuronales convolutivas

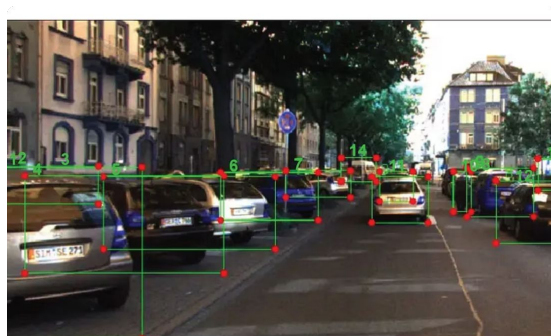
Las CNN se utilizan principalmente en tareas donde la entrada tiene una estructura espacial, siendo las imágenes el ejemplo más común.

Aplicaciones de las CNN



Clasificación de imágenes

Identificar si una imagen contiene un gato, perro, etc.



Detección de objetos

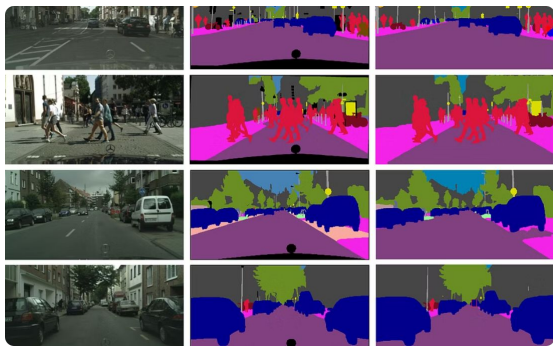
Localizar y clasificar objetos dentro de una imagen.



Reconocimiento facial

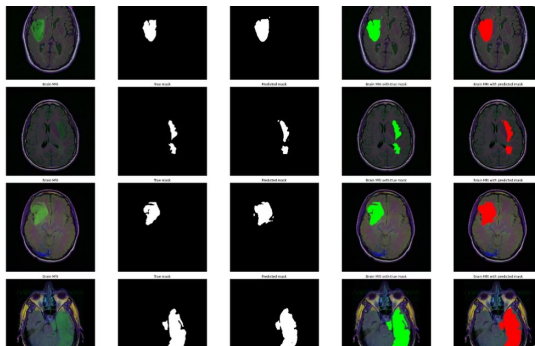
Autenticar usuarios mediante el análisis de rostros.

Más Aplicaciones de las CNN



Segmentación semántica

Clasificar cada píxel de una imagen según su clase.



Medicina

Detección de tumores en imágenes de resonancia o rayos X.



Automoción

Visión computacional para conducción autónoma.

Impacto de las CNN

Su capacidad para detectar patrones visuales ha convertido a las CNN en la arquitectura estándar en visión por computadora, reemplazando muchas técnicas tradicionales basadas en ingeniería manual de características.



Etapas en las RNC: operación de convolución y capa ReLU

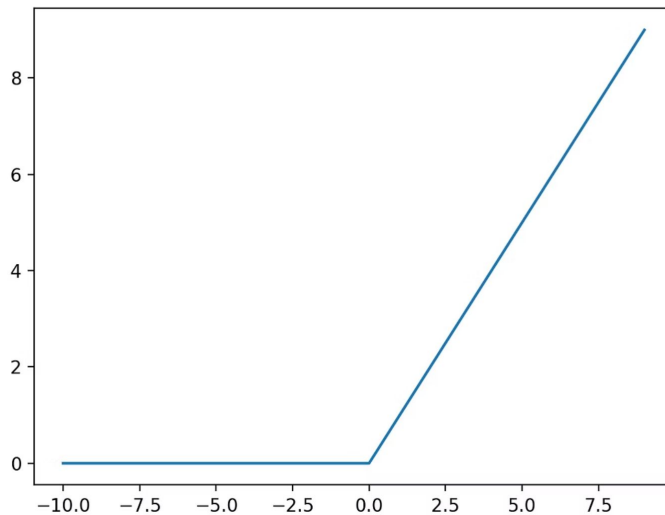
Una CNN se compone de varias etapas clave que permiten transformar progresivamente la imagen de entrada hasta generar una predicción:

1. Operación de convolución: como se explicó, aplica filtros para extraer características locales.
2. Capa ReLU (Rectified Linear Unit): introduce no linealidad, eliminando valores negativos y acelerando el entrenamiento.

Función de Activación ReLU

La fórmula de ReLU es: $f(x) = \max(0, x)$

Esta etapa permite que la red aprenda relaciones no lineales entre los datos. En conjunto, las capas de convolución y ReLU generan mapas de activación que resaltan las regiones más relevantes de la imagen.



Importancia de la Combinación Convolución + ReLU

Múltiples combinaciones de convolución + ReLU permiten detectar características cada vez más complejas, lo cual es esencial para tareas avanzadas como reconocimiento de escenas o clasificación por contexto.

Capa de pooling

El **pooling** es una operación que reduce la dimensión espacial de los mapas de activación, manteniendo las características más importantes. El tipo más común es el **Max Pooling**, que toma el valor máximo dentro de una ventana deslizando.

Beneficios del pooling

Reducción de parámetros

Disminuye significativamente la cantidad de parámetros que la red debe procesar.

Eficiencia computacional

Disminuye el tiempo de entrenamiento al reducir el tamaño de los mapas de características.

Prevención de sobreajuste

Ayuda a evitar el sobreajuste al reducir la complejidad del modelo.

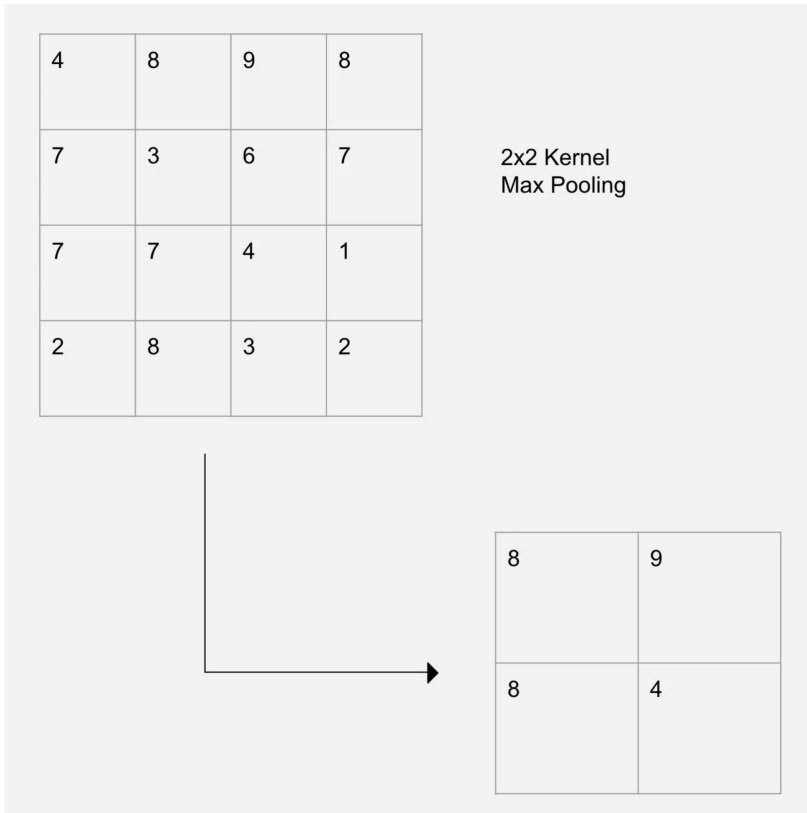
Invariancia espacial

Proporciona invariancia a pequeñas transformaciones espaciales en la imagen.

Ejemplo de Max Pooling

Por ejemplo, un Max Pooling 2x2 reduce una imagen de 32x32 a 16x16, manteniendo lo más destacado en cada región.

Esta reducción de dimensionalidad es crucial para hacer que la red sea computacionalmente eficiente mientras mantiene la información más relevante.





Flattening y Full Connection

Una vez procesadas las imágenes por capas convolutivas y de pooling, es necesario convertir los mapas de activación en un vector unidimensional. Esto se conoce como **flattening**.

Capas Fully Connected

Este vector es luego pasado a una o más capas densas (**fully connected layers**), donde cada neurona está conectada a todas las neuronas de la capa anterior. Estas capas se encargan de realizar la clasificación o regresión final.

Etapas Finales de la CNN

Etapas	Función principal
Flatten	Aplanar los mapas de activación
Full Connection	Interpretar características y generar salida

Las capas densas son similares a las usadas en redes neuronales tradicionales, y suelen ubicarse al final de las CNN como paso final de decisión.

Implementación de redes neuronales convolutivas en Python

Utilizando **Keras**, podemos construir una CNN de forma sencilla. A continuación, un ejemplo para clasificar imágenes de dígitos (dataset MNIST):

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

modelo = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax') # 10 clases
])
modelo.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```


Explicación del Código

1

Importación de Librerías

Se importan los módulos necesarios de Keras para construir la red neuronal.

2

Definición del Modelo

Se crea un modelo secuencial y se añaden las capas convolutivas, de pooling y densas.

3

Compilación

Se configura el optimizador, la función de pérdida y las métricas para el entrenamiento.

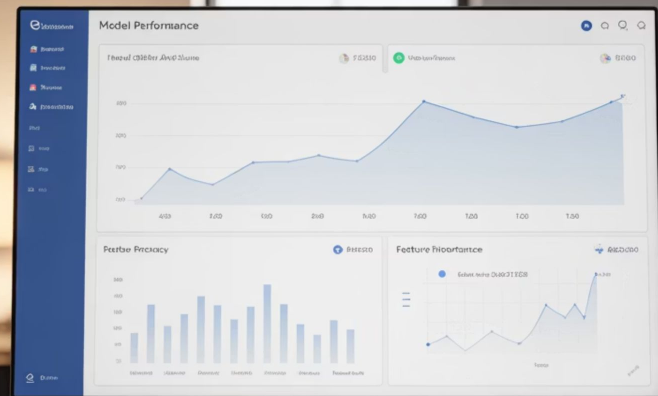
4

Entrenamiento y Evaluación

Se entrena el modelo con los datos y se evalúa su rendimiento.

Estructura del Modelo

Este modelo puede entrenarse con imágenes en blanco y negro de tamaño 28x28. La red extrae características espaciales, las convierte en un vector y luego predice la clase.



Métricas de Evaluación

1

Accuracy

Porcentaje de predicciones correctas. Es la métrica más intuitiva pero puede ser engañosa en datasets desbalanceados.

2

Confusion Matrix

Análisis detallado por clase que muestra verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos.

3

Precision / Recall / F1-score

Métricas más robustas que consideran diferentes aspectos del rendimiento del modelo.

Live Coding

¿En qué consistirá la Demo?

Explicaremos los pasos conceptuales para implementar una red neuronal convolutiva en Python que pueda reconocer imágenes.

1. Cómo se representan los datos de imagen (tensores RGB).
2. Diseño de la arquitectura CNN:
 - a. Capas convolutivas con filtros para extraer características.
 - b. Función de activación ReLU.
 - c. Capas de pooling para reducir dimensionalidad.
 - d. Capas fully connected para la clasificación final.
3. Elección de función de pérdida y optimizador (ej. cross-entropy y Adam).
4. Definición de métricas de evaluación (accuracy, precision, recall, F1).
5. Explicación del flujo: entrenamiento → validación → predicción.



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Diseña e implementa tu primera CNN

Diseña e implementa tu primera CNN

Contexto: 🙌

Vas a aplicar los conceptos de CNN para implementar tu propio modelo de reconocimiento de imágenes en Python.

Consigna: 📝

1. Elige un dataset de imágenes (ej. MNIST, CIFAR-10 o similar).
2. Diseña la arquitectura de una CNN:
3. Define cuántas capas convolutivas y de pooling tendrá.
4. Selecciona funciones de activación y fully connected final.
5. Configura función de pérdida, optimizador y métricas.
6. Entrena el modelo y valida su desempeño.
7. Genera predicciones y analiza los resultados obtenidos.

Paso a paso: ⚙️

1. Prepara el dataset y normaliza las imágenes.
2. Implementa la red CNN en Python con Keras o PyTorch.
3. Evalúa el modelo en datos no vistos y analiza métricas.
4. Ajusta la arquitectura o hiperparámetros si es necesario.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Aprendimos a diseñar e implementar una CNN en Python.**
- ✓ **Diseñamos arquitecturas capaces de reconocer imágenes.**
- ✓ **Configuramos funciones de activación, pérdidas y métricas para modelos de clasificación.**
- ✓ **Entrenamos y validamos una CNN utilizando datasets de imágenes.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

